

Web开发技术基础 期末复习资料整理

- Web开发技术基础 期末复习资料整理
 - 第一讲
 - Web开发基础
 - 3. 开发环境
 - 4. Web开发概念
 - 7. Web的基本结构与工作过程
 - 8. Web技术核心概念
 - 9. Web浏览器
 - 10. Web服务器
 - 第二讲
 - 1. IP地址
 - 2. 域名系统 (DNS)
 - 3. 域名解析
 - 4. 统一资源定位符URL
 - 5. 超文本传输协议HTTP
 - 6. HTTP数据包拆解分析
 - 7. HTTP事务剖析
 - 8. HTTP发展历史
 - 第三讲
 - 1. 超文本标记语言HTML
 - 2. 万维网联盟W3C
 - 3. Web标准
 - 4. HTML简介
 - 5. 学习HTML
 - 6. HTML元素
 - 7. 块级元素与内联元素
 - 8. HTML文档结构
 - 9. HTML文档头部
 - 10. HTML文档主体
 - 11. 段落与文字
 - 12. 文本格式化标签
 - 13. 列表
 - 第四讲
 - 1. HTML文档主体 (<body>)

- 2. 列表
- 3. 表格
- 4. 多媒体与嵌入
- 5. 超链接
- 6. 表单
- 第五讲
 - 1. 表单元素
 - 2. 文档与网站架构
 - 3. HTML验证
 - 4. CSS基础
 - 5. CSS引用方式
 - 6. CSS选择器
- 第六讲
 - 1. 伪类和伪元素选择器
 - 状态伪类
 - 结构伪类
 - 区别伪类
 - 验证伪类
 - 语言伪类
 - 2. 伪元素选择器
 - ::before 与 ::after
 - ::first-letter
 - ::first-line
 - ::selection
 - 3. 组合选择器和多重选择器
 - 群组选择器
 - 后代选择器
 - 子元素选择器
 - 相邻兄弟选择器
 - 后继兄弟选择器
 - 4. 通配选择器
 - *
 - 5. 选择器小结
 - 常见CSS选择器
 - 6. CSS样式概览
 - 文本和段落样式
- 第七讲
 - 1. 接下来主要内容

- 2. 文本和段落样式
 - 颜色 (color)
 - 文本装饰 (text-decoration)
 - 文本转换 (text-transform)
 - 文本阴影 (text-shadow)
 - 文本布局风格
- 3. 列表样式
 - list-style-type
 - list-style-position
 - list-style-image
- 4. 链接样式
 - 链接状态
 - 去除下划线
 - 添加图标
 - cursor属性
- 5. 图片样式
 - 宽度和高度
 - 边框
 - 对齐
 - 环绕效果
- 6. 背景样式
 - 背景颜色
 - 背景图像
 - 背景重复
 - 背景位置
 - 背景附着
- 7. 表格样式
 - 边框合并
 - 边框间距
- 8. 有关CSS的属性值
 - 关键字、数值、函数
- 第八讲
 - 1. 层叠和继承
 - 2. 层叠优先级算法
 - 3. 用户样式表
 - 4. 盒模型 (Box Model)
 - 5. 块级元素与内联元素
 - 6. 溢出 (Overflow)

- 7. 外边距重叠 (Collapsing Margins)
- 8. 自动外边距 (Auto Margins)
- 9. 内联元素的盒子模型
- 10. 内联块元素的盒子模型
- 11. 边框技巧
- 第九讲
 - 1. CSS布局概述
 - 2. 正常文档流
 - 3. 脱离正常文档流
 - 4. 浮动 (float)
 - 5. 定位
 - 6. display属性
 - 7. Flexbox布局
 - 8. Grid布局
 - 10. JavaScript导论
 - 11. 学习JavaScript的优势
 - 12. JavaScript语言的历史
- 第十讲
 - 1. JavaScript导论
 - 2. JavaScript在HTML中的引用方式
 - 3. 浏览器加载JavaScript的过程
 - 4. 从编译原理的角度学习基本语法
 - 5. 理解JavaScript的编译过程
 - 6. JavaScript引擎
 - 7. JavaScript试验环境
 - 8. JS主要内容
 - 9. 变量
 - 10. 条件语句
 - 11. 循环语句
 - 12. 数据类型简介
 - 13. typeof运算符
 - 14. null和undefined
 - 15. 布尔值
- 第十一讲
 - 1. 数值
 - 2. 字符串
 - 3. 对象
 - 4. 函数

- 5. 作用域
- 第十二讲
 - 1. 函数名的提升
 - 2. 函数的name属性
 - 3. 函数的length属性
 - 4. 函数的toString方法
 - 5. 函数作用域
 - 6. 函数参数
 - 7. 参数的两种传递方式
 - 8. arguments对象
 - 9. 数组的定义和本质
 - 10. 数组的length属性
 - 11. 数组的遍历
 - 12. 数组的空位
 - 13. 运算符
 - 14. 数据类型的转换
 - 15. 标准内置对象
- 第十三讲
 - 1. URI编解码函数
 - 2. 字符串对象String
 - 3. 日期对象Date
 - 4. 数组对象Array
 - 5. Math对象
 - 6. JavaScript与DOM
- 第十四讲
 - DOM节点操作
 - DOM节点接口和对象
 - 事件
 - 事件类型

第一讲

Web开发基础

- **Web**：全球信息共享平台，基于客户端-服务器架构。
- **HTML**：用于网页结构和内容标记。

- **CSS**：用于网页样式和布局设计。
- **JavaScript**：实现网页动态交互和行为。

3. 开发环境

- **集成开发环境 (IDE)**：如Visual Studio Code、WebStorm，用于编写和管理代码。
- **现代浏览器**：如Chrome、Firefox，用于测试和调试网页。

4. Web开发概念

- **前端开发**：
 - 技术：HTML、CSS、JavaScript。
 - 职责：网页内容展示、用户界面设计、交互实现。
- **后端开发**：
 - 技术：Java、Go、Python、Ruby。
 - 数据库：MySQL、MongoDB。
 - 职责：数据处理、业务逻辑、服务器管理。

7. Web的基本结构与工作过程

- **互联网与万维网**：
 - **互联网**：全球互联的计算机网络。
 - **万维网 (WWW)**：基于互联网的信息系统，使用HTTP协议。
- **Web的基本结构**：
 - **浏览器**：客户端，用于请求和展示网页。
 - **服务器**：存储网页资源，响应浏览器请求。
 - **IP与域名**：通过DNS将域名解析为IP地址。
 - **URL**：统一资源定位符，指定资源地址。
 - **HTTP**：超文本传输协议，定义通信规则。
- **Web的工作过程**：

1. 用户在浏览器输入URL。
2. 浏览器通过DNS解析域名获取服务器IP。
3. 浏览器发送HTTP请求到服务器。
4. 服务器处理请求并返回HTTP响应。
5. 浏览器渲染页面内容。

8. Web技术核心概念

- URL:

```
https://www.example.com:443/path/page.html?query=123#section
```

- **组成**: 协议 (https)、主机 (www.example.com)、端口 (443)、路径 (/path/page.html)、查询参数 (?query=123)、片段标识符 (#section)。

- HTTP:

- **特点**: 无状态、无连接、面向请求-响应。
- **方法**: GET、POST、PUT、DELETE。

- HTML:

- **作用**: 定义网页结构和内容。
- **基本标签**: `<html>`, `<head>`, `<body>`, `<div>`, `<span>`。

9. Web浏览器

- 浏览器结构:

- **用户界面**: 地址栏、标签页、书签等。
- **浏览器引擎**: 协调UI与渲染引擎。
- **渲染引擎**: 解析HTML/CSS, 渲染页面 (如Blink)。
- **网络**: 处理HTTP请求。
- **JS解释器**: 执行JavaScript代码。
- **数据持久层**: 存储缓存、Cookie等数据。

- 主要Web浏览器发布时间:

- **WorldWideWeb** (1990)
- **Mosaic** (1993)
- **Netscape Navigator** (1994)
- **Internet Explorer** (1995)
- **Chrome** (2008)
- **Firefox** (2002)
- **Safari** (2003)
- **Edge** (2015)

10. Web服务器

- **服务器结构：**
 - 存储和管理网页文件。
 - 处理客户端HTTP请求，返回响应。
 - **市场份额：**
 - **Apache**：全球使用最广的Web服务器。
 - **nginx**：高性能、支持高并发。
 - **Microsoft IIS**：集成于Windows服务器环境。
 - **C10X问题：**
 - 单台服务器处理大量并发连接的挑战，解决方案包括负载均衡和服务器集群。
-

第二讲

1. IP地址

- **定义：** 每台连接互联网的设备的唯一标识，IPv4为32位，格式如192.168.1.1。
- **构成：** 网络号 (Netid) + 主机号 (Hostid) 。
- **分类：**
 - **A类：** 1.0.0.0 - 126.255.255.255
 - **B类：** 128.0.0.0 - 191.255.255.255
 - **C类：** 192.0.0.0 - 223.255.255.255
 - **D类：** 224.0.0.0 - 239.255.255.255 (多播)

- E类：240.0.0.0 - 255.255.255.255（保留）
- 私有网IP地址：如192.168.x.x，用于局域网，不需全球唯一。

2. 域名系统（DNS）

- 定义：将域名转换为IP地址的系统。
- 组成：
 - DNS域名空间：分层结构。
 - DNS服务器：负责解析请求。
 - DNS客户端：发起解析请求。
 - 资源记录：存储域名与IP的映射。
- 域名结构：根域、顶级域（如.com）、二级域、子域、主机名。
- 顶级域名：
 - 组织模式：.com（商业）、.edu（教育）、.gov（政府）。
 - 地理模式：.cn（中国）、.jp（日本）、.uk（英国）。

3. 域名解析

- 过程：将用户输入的域名通过DNS查询转换为IP地址。
- 模式：
 - 递归查询：DNS服务器递归查找直到找到IP。
 - 迭代查询：DNS服务器提供部分信息，客户端继续查询。

4. 统一资源定位符URL

- 定义：指定资源位置和访问方法的地址。
- 构成：
 - 协议：如http、https。
 - 主机名：如www.example.com。
 - 端口号：默认80（HTTP）、443（HTTPS）。
 - 路径：资源在服务器上的位置。
 - 文件名：具体文件，如index.html。

5. 超文本传输协议HTTP

- 定义：客户端与服务器之间的通信协议，基于TCP。

- **特点：**无状态、无连接、面向请求-响应。
- **报文结构：**
 - **请求报文：**方法、URL、HTTP版本、头部、主体。
 - **响应报文：**状态行、头部、主体。
- **请求方法：**
 - **GET：**请求资源。
 - **POST：**提交数据。
 - **PUT：**更新资源。
 - **DELETE：**删除资源。
- **状态码：**
 - **1xx：**信息性。
 - **2xx：**成功。
 - **3xx：**重定向。
 - **4xx：**客户端错误。
 - **5xx：**服务器错误。

6. HTTP数据包拆解分析

- **请求：**

```
GET /index.html HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept: text/html
```

- **响应：**

```
HTTP/1.1 200 OK
Date: Mon, 27 Jul 2023 12:28:53 GMT
Content-Type: text/html
Content-Length: 137
Connection: keep-alive

<html>
  <body>
    <h1>Hello, World!</h1>
  </body>
</html>
```

7. HTTP事务剖析

- **简单HTTP事务：**
 1. DNS查找。
 2. 建立TCP连接。
 3. 发送HTTP请求。
 4. 服务器响应。
 5. 加载内容。
 6. 关闭TCP连接。

8. HTTP发展历史

- **版本特点：**
 - **HTTP/0.9：**仅支持GET方法，纯文本。
 - **HTTP/1.0：**支持状态码、头部字段。
 - **HTTP/1.1：**持久连接、管道化请求。
 - **HTTP/2：**多路复用、头部压缩。
 - **HTTP/3：**基于QUIC协议，减少延迟。
-

第三讲

1. 超文本标记语言HTML

- **定义：**用于结构化网页内容的标记语言。
- **功能：**通过标签描述文本、图片、链接等元素，实现内容组织和导航。

2. 万维网联盟W3C

- **创建与目的：**1994年由Tim Berners-Lee创建，标准化Web技术。
- **工作：**制定和维护Web标准，如HTML、CSS、WCAG。

3. Web标准

- **组成：**
 - **结构：**HTML、XML、XHTML。
 - **呈现：**CSS。
 - **行为：**DOM、ECMAScript (JavaScript) 。

4. HTML简介

- **定义：**使用标签定义网页内容和结构，支持超链接。
- **发展历程：**从HTML2.0到HTML5，引入新标签和功能。

5. 学习HTML

- **重点：**
 - HTML元素（标签）的正确使用。
 - 语义化标签，如<header>、<article>。
 - 搜索引擎优化（SEO）相关设计。

以下是对你HTML笔记中代码的注释：

6. HTML元素

- **结构：**

```
<element attribute="value">内容</element>
```

- **<element>**：定义了HTML的基本标签（例如<div>、<p>）。
- **attribute="value"**：属性是用来提供关于元素的附加信息，如**class**、**id**等，控制样式或行为。
- **内容**：元素包含的内容，可以是文字、图像等其他HTML元素。

7. 块级元素与内联元素

- **块级元素：**

```
<div>内容</div>
```

- **div**: 块级元素, 独占一行, 并且可以设置宽高、边距等样式。
- 其他块级元素: `<p>`、`<h1>`-`<h6>`等。

- **内联元素:**

```
<span>内容</span>
```

- **span**: 内联元素, 和其他内容在同一行内显示, 通常用于文字样式或局部内容的包装。
- 其他内联元素: `<a>`、`<img>`等。

8. HTML文档结构

- **组成:**

```
<!DOCTYPE html> <!-- 声明文档类型, 告知浏览器使用HTML5标准 -->
<html> <!-- HTML文档的根元素 -->
  <head> <!-- 包含页面的元信息 -->
    <title>页面标题</title> <!-- 网页标题, 显示在浏览器标签页上 -->
  </head>
  <body> <!-- 页面主要内容 -->
    <!-- 页面内容 -->
  </body>
</html>
```

- `<!DOCTYPE html>`: 告诉浏览器使用HTML5标准解析文档。
- `<html>`: HTML文档的根标签, 包含所有页面内容。
- `<head>`: 文档的元数据部分, 包含如标题、CSS样式和JavaScript等资源的引入。
- `<body>`: 页面的主体内容部分, 所有网页内容都应放在这里。

9. HTML文档头部

- **功能:**

- 元信息: `<meta>`标签用于指定字符集、作者、描述等信息。

```
<meta charset="UTF-8"> <!-- 设置字符集为UTF-8 -->
<meta name="description" content="网页描述"> <!-- 设置网页描述，用于SEO -->
```

- 标题：<title>设置网页在浏览器标签上的显示标题。

```
<title>网页标题</title>
```

- 引入外部资源：<link>用于引入CSS样式表，<script>用于引入JavaScript文件。

```
<link rel="stylesheet" href="styles.css"> <!-- 引入CSS样式 -->
<script src="script.js"></script> <!-- 引入JavaScript文件 -->
```

10. HTML文档主体

- 内容：

- 段落与文字：

```
<p>这是一个段落。</p> <!-- 段落标签，用于定义段落 -->
<h1>主标题</h1> <!-- 标题标签，表示主标题，显示在页面最上方 -->
<h2>副标题</h2> <!-- 标题标签，表示副标题 -->
```

- 列表：

- 有序列表：

```
<ol>
  <li>第一项</li>
  <li>第二项</li>
</ol> <!-- 有序列表，自动生成编号 -->
```

- 无序列表：

```
<ul>
  <li>项目一</li>
```

```
<li>项目二</li>
</ul> <!-- 无序列表，显示圆点或其他符号 -->
```

■ 描述列表：

```
<dl>
  <dt>HTML</dt> <!-- 定义术语 -->
  <dd>超文本标记语言</dd> <!-- 定义术语的描述 -->
</dl> <!-- 描述列表，用于定义术语及其描述 -->
```

○ 表格：

```
<table>
  <tr>
    <td>单元格内容</td> <!-- 表格中的单元格 -->
  </tr>
</table> <!-- 表格标签，用于展示表格数据 -->
```

○ 多媒体与嵌入：

```
 <!-- 图片标签，显示图片 -->
<video src="video.mp4" controls> <!-- 视频标签，允许播放控制 -->
  <source src="video.mp4" type="video/mp4">
</video>
<audio src="audio.mp3" controls> <!-- 音频标签，允许音频播放 -->
</audio>
```

○ 超链接：

```
<a href="https://www.example.com">点击访问</a> <!-- 超链接标签，用于创建跳转到其他页面或地址的链接 -->
```

○ 表单：

```
<form action="/submit" method="POST">
  <input type="text" name="username"> <!-- 输入框，允许用户输入文本 -->
  <textarea name="comments"></textarea> <!-- 多行文本框 -->
  <input type="submit" value="提交"> <!-- 提交按钮 -->
</form> <!-- 表单标签，提交数据到服务器 -->
```

11. 段落与文字

- 标题标签：

```
<h1>主标题</h1> <!-- 主标题，通常用于页面的主要标题 -->
<h2>副标题</h2> <!-- 副标题，通常是次要标题 -->
```

- 段落标签：

```
<p>这是一个段落。</p> <!-- 段落标签，用于显示一段文字内容 -->
<br/> <!-- 换行标签，用于强制换行 -->
```

12. 文本格式化标签

- 常用标签：

```
<em>强调</em> <!-- 强调文本，通常以斜体显示 -->
<strong>重要</strong> <!-- 强调文本，通常以加粗显示 -->
<sup>上标</sup> <!-- 上标文本，显示在基线之上 -->
<sub>下标</sub> <!-- 下标文本，显示在基线之下 -->
<time datetime="2024-04-27">April 27, 2024</time> <!-- 表示日期时间 -->
<abbr title="World Health Organization">WHO</abbr> <!-- 缩写标签，提供详细解释 -->
<address>联系地址</address> <!-- 地址标签，显示联系地址 -->
```

13. 列表

- 有序列表：

```
<ol>
  <li>第一项</li>
  <li>第二项</li>
</ol> <!-- 有序列表，显示项目的编号 -->
```

- 无序列表：


```
<ul>
  <li>项目一</li>
  <li>项目二</li>
</ul> <!-- 无序列表，显示带符号（通常是圆点）的项目 -->
```

- **描述列表：**

```
<dl>
  <dt>HTML</dt> <!-- 定义术语 -->
  <dd>超文本标记语言</dd> <!-- 对术语的描述 -->
</dl> <!-- 描述列表，用于定义一系列术语及其描述 -->
```

第四讲

1. HTML文档主体（<body>）

- **包含：**
 - 段落与文字
 - 列表
 - 表格
 - 多媒体与嵌入
 - 超链接
 - 表单

以下是你提供的HTML代码注释版本，解释了每个元素的渲染效果：

2. 列表

- **有序列表：**有序列表展示的是一个数字或字母顺序的列表，使用标签，是每一项的内容。

```
<ol>
  <li>项目1</li> <!-- 第一项，自动按顺序编号 -->
  <li>项目2</li> <!-- 第二项，继续按顺序编号 -->
</ol>
```

- **无序列表**：无序列表显示的项前会有小圆点，使用标签。

```
<ul>
  <li>项目A</li> <!-- 每个列表项前有一个圆点 -->
  <li>项目B</li> <!-- 第二项，前面有圆点 -->
</ul>
```

- **描述列表**：描述列表通常用于定义术语和它们的定义。<dt>用于定义术语，<dd>用于定义术语的解释。

```
<dl>
  <dt>术语</dt> <!-- 术语定义 -->
  <dd>定义</dd> <!-- 术语的解释 -->
</dl>
```

3. 表格

- **基本元素**：表格使用<table>标签，表头使用<th>，表格行使用<tr>，单元格使用<td>。

```
<table>
  <tr>
    <th>标题1</th> <!-- 表头，通常用来描述每列的内容 -->
    <th>标题2</th> <!-- 表头的另一列 -->
  </tr>
  <tr>
    <td>数据1</td> <!-- 表格数据 -->
    <td>数据2</td> <!-- 另一列数据 -->
  </tr>
</table>
```

- **标题**：<caption>用于为表格添加标题。

```
<table>
  <caption>这是表格的标题</caption> <!-- 表格标题 -->
  <tr>
    <th>标题1</th>
    <th>标题2</th>
  </tr>
  <tr>
    <td>数据1</td>
    <td>数据2</td>
```

```
</tr>
</table>
```

- **跨行跨列：** `colspan`用于合并列，`rowspan`用于合并行。

```
<td colspan="2">跨两列</td> <!-- 单元格横跨两列 -->
<td rowspan="3">跨三行</td> <!-- 单元格纵跨三行 -->
```

- **语义化：** `<thead>`、`<tbody>`、`<tfoot>`分别表示表格的表头、表体和表尾，增强表格的语义化。

```
<thead>表头</thead> <!-- 表格的标题部分 -->
<tbody>表体</tbody> <!-- 表格的主要数据部分 -->
<tfoot>表尾</tfoot> <!-- 表格的尾部，通常包含总结或合计 -->
```

4. 多媒体与嵌入

- **图片：** `<img>`标签用于显示图片，`src`指定图片路径，`alt`提供图片的替代文本，`width`和`height`设置图片的显示大小。

```
 <!-- 显示图片，设定大小 -->
```

- **视频与音频：** `<video>`和`<audio>`标签用于嵌入视频和音频文件，`controls`属性添加控制按钮，如播放、暂停等。

```
<video controls>
  <source src="video.mp4" type="video/mp4"> <!-- 视频文件 -->
</video>

<audio controls>
  <source src="audio.mp3" type="audio/mpeg"> <!-- 音频文件 -->
</audio>
```

- **通用嵌入：** `<embed>`用于嵌入外部资源，如PDF文件；`<object>`可以嵌入多种文件类型（例如SWF）。

```
<embed src="file.pdf" type="application/pdf"/> <!-- 嵌入PDF文件 -->
<object data="file.swf" type="application/x-shockwave-flash"></object> <!-- 嵌入SWF文件 -->
```

- **页面嵌入：** `<iframe>` 标签用于在当前页面中嵌入另一个网页，`src` 指定嵌入页面的 URL，`width` 和 `height` 设定显示区域的大小。

```
<iframe src="https://www.example.com" width="600" height="400"></iframe> <!-- 嵌入外部网页 -->
```

5. 超链接

- **创建：** `<a>` 标签用于创建超链接，`href` 指定链接的目标地址。

```
<a href="https://www.example.com">访问示例</a> <!-- 点击后跳转到指定网址 -->
```

- **属性：**

- `title`：为链接提供提示信息。
- `target`：控制链接在何处打开，`_blank` 表示在新标签页中打开。
- `download`：使链接成为下载链接，点击即可下载文件。

```
<a href="https://www.example.com" title="示例链接" target="_blank">访问示例</a>
<!-- 点击后在新标签页打开链接 -->
```

- **锚点链接：** 用于在页面内部跳转。`href` 指向页面内的 ID。

```
<a href="#section1">跳转到Section 1</a> <!-- 点击后跳转到页面内的Section 1 -->
<h2 id="section1">Section 1</h2> <!-- 目标位置 -->
```

- **电子邮件链接：** 通过 `mailto:` 协议创建邮件链接，点击后会弹出邮件客户端。

```
<a href="mailto:someone@example.com">发送邮件</a> <!-- 点击后打开默认邮件客户端 -->
```

- **块级链接** (HTML5) : `<a>`可以包裹块级元素 (如`<div>`) , 使整个区域成为可点击链接。

```
<a href="https://www.example.com">
  <div>块级内容</div> <!-- 点击整个块级内容跳转 -->
</a>
```

6. 表单

- **定义**: 表单通过`<form>`标签定义, `action`指定提交数据的URL, `method`指定提交数据的方式 (如`post`或`get`) 。

```
<form action="/submit" method="post">
  <input type="text" name="username"/> <!-- 用户名输入框 -->
  <button type="submit">提交</button> <!-- 提交按钮 -->
</form>
```

- **表单对象**:
 - `<input>`: 定义多种类型的输入框, 如文本框、密码框、单选框、复选框等。
 - `<textarea>`: 定义多行文本输入框。
 - `<select>` 和 `<option>`: 用于创建下拉选择框。
 - `<datalist>`: 提供一个自动补全的输入框。
 - `<fieldset>` 和 `<legend>`: 用于分组表单元素, `<legend>`提供描述。
 - `<label>`: 用于描述表单控件, 增加可访问性。

```
<input type="text" name="username"/> <!-- 文本输入框 -->
<textarea name="message"></textarea> <!-- 多行文本框 -->
<select name="options">
  <option value="1">选项1</option> <!-- 下拉菜单项 -->
  <option value="2">选项2</option> <!-- 下拉菜单项 -->
</select>
```

第五讲

1. 表单元素

- **<textarea>**:

```
<textarea rows="4" cols="50">请输入内容...</textarea>
```

- **<select> 和 <option>**:

```
<select name="fruits" multiple>  
  <option value="apple">苹果</option>  
  <option value="banana">香蕉</option>  
</select>
```

- **<datalist>**:

```
<input list="browsers" name="browser"/>  
<datalist id="browsers">  
  <option value="Chrome">  
  <option value="Firefox">  
</datalist>
```

- **<fieldset> 和 <legend>**:

```
<fieldset>  
  <legend>个人信息</legend>  
  <label>姓名:</label>  
  <input type="text" name="name"/>  
</fieldset>
```

- **<label>**:

```
<label for="email">邮箱:</label>  
<input type="email" id="email" name="email"/>
```

2. 文档与网站架构

- **基本组成**: 页眉 (**<header>**)、导航栏 (**<nav>**)、主内容 (**<main>**)、页脚 (**<footer>**)、侧边栏 (**<aside>**)。
- **语义化标记**:

```
<header>网站头部</header>
<nav>导航菜单</nav>
<main>
  <article>文章内容</article>
  <section>章节</section>
</main>
<aside>侧边栏内容</aside>
<footer>网站底部</footer>
```

- **规划简单网站：**
 - 内容规划：确定网站主题和内容。
 - 页面结构草图：绘制页面布局。
 - 内容分组：将内容分类组织。
 - 站点地图草图：规划网站结构和链接。

3. HTML验证

- **Markup Validation Service：**使用W3C的验证工具检查HTML文档的语法错误。

访问 <https://validator.w3.org/>，上传HTML文件或输入URL进行验证。

4. CSS基础

- **定义：**层叠样式表，描述HTML或XML文档的呈现。
- **发展历史：**
 - **CSS1：**基础样式。
 - **CSS2.1：**布局和媒体类型。
 - **CSS3：**模块化发展，新增动画、响应式设计等。
- **工作原理：**
 - **DOM树：**HTML解析为DOM树。
 - **CSS规则应用：**根据选择器匹配，将样式应用到元素。

5. CSS引用方式

- **外部样式表：**

```
<link rel="stylesheet" href="styles.css"/>
```

- 内部样式表：

```
<style>  
  body { background-color: #f0f0f0; }  
</style>
```

- 内联样式：

```
<p style="color: blue;">蓝色文字</p>
```

6. CSS选择器

- 简单选择器：

- 元素选择器：

```
p { font-size: 16px; }
```

- 类选择器：

```
.highlight { background-color: yellow; }
```

- ID选择器：

```
#header { height: 60px; }
```

- 属性选择器：

```
a[href^="https"] { color: green; }
```

- 伪类和伪元素选择器：

- 伪类：:hover、:nth-child(2)。

- 伪元素：`::before`、`::after`。

第六讲

1. 伪类和伪元素选择器

状态伪类

```
a:hover { color: red; } /* 鼠标悬停时改变链接颜色为红色 */
```

结构伪类

```
li:first-child { font-weight: bold; } /* 选择每个列表的第一个项并加粗 */
```

区别伪类

```
:nth-child(odd) { background-color: #f9f9f9; } /* 选择奇数位置的元素并设置背景色 */
```

验证伪类

```
input:valid { border-color: green; } /* 输入合法时设置绿色边框 */  
input:invalid { border-color: red; } /* 输入非法时设置红色边框 */
```

语言伪类

```
p:lang(en) { font-style: italic; } /* 选择英语段落并设置斜体 */
```

2. 伪元素选择器

::before 与 ::after

```
.button::before { content: '★ '; } /* 按钮前插入星号 */  
.button::after { content: ' ★'; } /* 按钮后插入星号 */
```

::first-letter

```
p::first-letter { font-size: 2em; } /* 选择段落的首字母并放大 */
```

::first-line

```
p::first-line { text-transform: uppercase; } /* 选择段落的首行并转换为大写 */
```

::selection

```
::selection { background: yellow; } /* 选中文本的背景设置为黄色 */
```

3. 组合选择器和多重选择器

群组选择器

```
h1, h2, h3 { color: blue; } /* 同时设置h1、h2、h3元素的文本颜色为蓝色 */
```

后代选择器

```
div p { margin-bottom: 10px; } /* 设置div中的所有p元素的下边距 */
```

子元素选择器

```
ul > li { list-style-type: square; } /* 设置ul中的直接li子元素的列表样式为方块 */
```

相邻兄弟选择器

```
h1 + p { margin-top: 0; } /* 设置紧跟在h1后面的p元素的上边距为0 */
```

后继兄弟选择器

```
h1 ~ p { color: gray; } /* 设置h1后的所有p元素的文本颜色为灰色 */
```

4. 通配选择器

*

```
* { box-sizing: border-box; } /* 设置所有元素的box-sizing为border-box */
```

5. 选择器小结

常见CSS选择器

```
#id .class element:hover::before { /* 样式 */ } /* 示例：从高到低的选择器优先级 */
```

6. CSS样式概览

文本和段落样式

```
p {  
  font-family: Arial, sans-serif; /* 字体 */  
  font-size: 14px; /* 字体大小 */  
  font-weight: bold; /* 字体加粗 */  
  text-transform: uppercase; /* 转换为大写 */  
  color: #333; /* 字体颜色 */  
  text-decoration: underline; /* 添加下划线 */  
  text-shadow: 1px 1px 2px #aaa; /* 添加文本阴影 */  
}
```

```
text-indent: 20px; /* 设置文本缩进 */
text-align: justify; /* 两端对齐 */
line-height: 1.5; /* 行高 */
letter-spacing: 2px; /* 字母间距 */
word-spacing: 4px; /* 单词间距 */
}
```

第七讲

1. 接下来主要内容

- 选择器
- 样式
- 层叠和继承
- 盒模型
- 布局

2. 文本和段落样式

颜色 (color)

```
p { color: #ff0000; } /* 使用十六进制设置颜色为红色 */
p { color: rgb(255, 0, 0); } /* 使用RGB设置颜色为红色 */
p { color: rgba(255, 0, 0, 0.5); } /* 使用RGBA设置红色，带透明度 */
p { color: hsl(0, 100%, 50%); } /* 使用HSL设置红色 */
```

文本装饰 (text-decoration)

```
a { text-decoration: none; } /* 去除链接的下划线 */
```

文本转换 (text-transform)

```
p { text-transform: uppercase; } /* 将文本转为大写 */
```

文本阴影 (**text-shadow**)

```
h1 { text-shadow: 2px 2px 5px #000; } /* 为标题添加阴影 */
```

文本布局风格

```
p {  
  text-indent: 2em; /* 设置文本缩进 */  
  text-align: center; /* 设置文本居中 */  
  line-height: 1.6; /* 设置行高 */  
  letter-spacing: 0.1em; /* 设置字母间距 */  
  word-spacing: 0.2em; /* 设置单词间距 */  
}
```

3. 列表样式

list-style-type

```
ul { list-style-type: disc; } /* 设置无序列表为圆点符号 */  
ol { list-style-type: decimal; } /* 设置有序列表为数字 */
```

list-style-position

```
ul { list-style-position: inside; } /* 设置列表符号位于列表项内 */
```

list-style-image

```
ul { list-style-image: url('bullet.png'); } /* 使用自定义图像作为列表符号 */
```

4. 链接样式

链接状态

```
a:link { color: blue; } /* 设置未访问链接的颜色 */
a:visited { color: purple; } /* 设置已访问链接的颜色 */
a:hover { color: red; } /* 鼠标悬停时链接颜色为红色 */
a:active { color: orange; } /* 激活状态下的链接颜色 */
```

去除下划线

```
a { text-decoration: none; } /* 去除所有链接的下划线 */
```

添加图标

```
a::before {
  content: url('icon.png'); /* 在链接前添加图标 */
  margin-right: 5px; /* 图标与文字之间添加右边距 */
}
```

cursor属性

```
a:hover { cursor: pointer; } /* 鼠标悬停时，链接变为手型光标 */
```

5. 图片样式

宽度和高度

```
img { width: 100px; height: auto; } /* 设置图片宽度为100px，自动调整高度 */
```

边框

```
img { border: 2px solid #000; } /* 为图片添加2px的黑色边框 */
```

对齐

```
img { float: left; margin: 10px; } /* 设置图片浮动到左边，并添加10px的外边距 */
```

环绕效果

```
.text { text-align: wrap; } /* 使文本环绕图片（实际用法更改为 'text-align: justify'） */
```

6. 背景样式

背景颜色

```
body { background-color: #f0f0f0; } /* 设置页面背景颜色为灰色 */
```

背景图像

```
.header { background-image: url('header-bg.jpg'); } /* 为头部设置背景图像 */
```

背景重复

```
.pattern { background-repeat: repeat-x; } /* 设置背景图像只在水平轴上重复 */
```

背景位置

```
.banner { background-position: center top; } /* 设置背景图像位置为居中对齐，顶部对齐 */
```

背景附着

```
.fixed-bg { background-attachment: fixed; } /* 设置背景固定，不随页面滚动 */
```

7. 表格样式

边框合并

```
table { border-collapse: collapse; } /* 合并表格边框 */
```

边框间距

```
table { border-spacing: 10px; } /* 设置表格单元格的边距为10px */
```

8. 有关CSS的属性值

关键字、数值、函数

```
width: 100px; /* 设置宽度为100px */
margin: auto; /* 设置自动外边距，用于居中 */
padding: calc(10px + 5%); /* 使用calc函数计算边距，混合px和百分比 */
```

第八讲

1. 层叠和继承

- **层叠 (Cascade)**：多个CSS规则作用于同一元素时，根据权重决定应用哪条规则。
- **特殊性 (Specificity)**：
 - ID选择器 > 类选择器 > 元素选择器。
 - 内联样式 > 外部样式表。
- **样式继承 (Inheritance)**：子元素继承父元素的某些属性，如color、font-family。

2. 层叠优先级算法

1. 列出所有应用于元素的CSS规则。
2. 按声明的重要性（`!important`优先）。
3. 按选择器特殊性排序。
4. 按样式表中的出现顺序排序。

3. 用户样式表

- 用户可以通过浏览器设置自定义样式，覆盖网站的默认样式。

```
/* 用户样式表示例 */  
body { font-size: 18px !important; }
```

4. 盒模型 (Box Model)

- 组成：
 - 内容区 (Content)：显示实际内容。
 - 内边距 (Padding)：内容区与边框之间的空间。
 - 边框 (Border)：围绕内边距的线条。
 - 外边距 (Margin)：边框之外的空间，用于分隔其他元素。
- 示例：

```
.box {  
  width: 200px;  
  padding: 10px;  
  border: 2px solid #000;  
  margin: 20px;  
}
```

5. 块级元素与内联元素

- 块级元素：
 - 占据整行。
 - 可设置宽高。
 - 示例：<div>, <p>, <h1>.
- 内联元素：

- 与文本在同一行。
- 不可设置宽高。
- 示例：, <a>, .

- 内联块元素：

- 结合块级和内联元素特性。
- 可设置宽高，且在行内排列。
- 示例：

```
.inline-block {  
  display: inline-block;  
  width: 100px;  
  height: 50px;  
}
```

6. 溢出 (Overflow)

- 属性值：

- **visible**：默认，内容溢出显示。
- **hidden**：隐藏溢出内容。
- **scroll**：显示滚动条。
- **auto**：根据需要显示滚动条。

- 示例：

```
.container {  
  width: 300px;  
  height: 200px;  
  overflow: auto;  
}
```

7. 外边距重叠 (Collapsing Margins)

- 相邻块级元素的上下外边距会合并为较大的那个外边距。
- 示例：

```
.first { margin-bottom: 20px; }  
.second { margin-top: 30px; }
```

8. 自动外边距 (Auto Margins)

- 居中块级元素：

```
.center {  
  width: 50%;  
  margin-left: auto;  
  margin-right: auto;  
}
```

9. 内联元素的盒子模型

- 宽度和高度不可设置，但外边距、内边距和边框可用。

```
span {  
  padding: 5px;  
  border: 1px solid #000;  
}
```

10. 内联块元素的盒子模型

- 可设置宽度和高度，且外边距、内边距和边框推开其他元素。

```
.inline-block {  
  display: inline-block;  
  width: 100px;  
  height: 50px;  
  margin: 10px;  
  padding: 5px;  
  border: 1px solid #000;  
}
```

11. 边框技巧

- 三角形：

```
.triangle {  
  width: 0;  
  height: 0;  
  border-left: 10px solid transparent;  
  border-right: 10px solid transparent;  
  border-bottom: 20px solid #555;  
}
```

- 圆角边框：

```
.rounded {  
  border: 2px solid #000;  
  border-radius: 10px;  
}
```

第九讲

1. CSS布局概述

- 布局过程：
 1. 选择元素。
 2. 应用布局属性控制位置。
- 正常文档流：元素按顺序排列。
- 脱离正常文档流：使用float、position等属性。

2. 正常文档流

- 特点：
 - 元素依次排列。
 - 保持语义结构。
- 优势：即使CSS加载失败，内容仍可阅读。

3. 脱离正常文档流

- **float属性**：使元素浮动到左或右。
- **position属性**：控制元素定位。
- **display属性**：改变元素显示行为。

4. 浮动 (float)

- 使用：

```
.float-left { float: left; }  
.float-right { float: right; }
```

- 清除浮动：

```
.clearfix::after {  
  content: "";  
  clear: both;  
  display: table;  
}
```

5. 定位

- 定位类型：
 - 静态定位（默认）：按正常文档流排列。
 - 相对定位：

```
.relative { position: relative; top: 10px; left: 20px; }
```

- 绝对定位：

```
.absolute { position: absolute; top: 50px; left: 100px; }
```

- 固定定位：

```
.fixed { position: fixed; top: 0; right: 0; }
```

- 示例：

```
<div class="relative">相对定位</div>
<div class="absolute">绝对定位</div>
<div class="fixed">固定定位</div>
```

6. display属性

- **display: none**：隐藏元素。
- 下拉菜单和导航菜单：

```
.dropdown-menu { display: none; }
.dropdown:hover .dropdown-menu { display: block; }
```

7. Flexbox布局

- 定义：一维布局模型，适用于行或列排列元素。
- 示例：

```
.flex-container {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.flex-item {
  flex: 1;
  margin: 10px;
}
```

```
<div class="flex-container">
  <div class="flex-item">Item 1</div>
  <div class="flex-item">Item 2</div>
  <div class="flex-item">Item 3</div>
</div>
```

8. Grid布局

- **定义：**二维布局模型，按行和列排列元素。
- **示例：**

```
.grid-container {  
  display: grid;  
  grid-template-columns: repeat(3, 1fr);  
  gap: 10px;  
}  
.grid-item {  
  background-color: #ccc;  
  padding: 20px;  
  text-align: center;  
}
```

```
<div class="grid-container">  
  <div class="grid-item">1</div>  
  <div class="grid-item">2</div>  
  <div class="grid-item">3</div>  
  <div class="grid-item">4</div>  
  <div class="grid-item">5</div>  
  <div class="grid-item">6</div>  
</div>
```

10. JavaScript导论

- **定义：**轻量级脚本语言，用于实现网页动态功能。
- **标准：**ECMAScript。
- **用途：**前端交互、后端开发（如Node.js）。

11. 学习JavaScript的优势

- **操控浏览器：**内置于浏览器，直接操作DOM。
- **广泛应用：**前端、后端、全栈开发。
- **易学性：**语法简单，学习资源丰富。
- **性能强大：**即时编译（JIT）提升执行速度。
- **开放性：**遵循ECMA-262标准。
- **社区支持：**庞大开发者社区，丰富库和框架。

12. JavaScript语言的历史

- **起源**：由Brendan Eich设计，最初名为Mocha和LiveScript。
 - **与Java的关系**：语法模仿Java，但两者独立。
 - **与ECMAScript的关系**：ECMAScript是JavaScript的标准化版本。
-

第十讲

1. JavaScript导论

- **内容**：核心语法、标准库、浏览器API、DOM操作。

2. JavaScript在HTML中的引用方式

- **内联脚本**：

```
<script>  
  console.log('Hello, World!');  
</script>
```

- **事件属性**：

```
<button onclick="alert('Clicked!')">点击我</button>
```

- **外部文件**：

```
<script src="script.js"></script>
```

- **integrity**属性：

```
<script src="https://example.com/script.js" integrity="sha384-abc123"  
  crossorigin="anonymous"></script>
```

3. 浏览器加载JavaScript的过程

- 浏览器解析HTML，遇到<script>暂停解析，下载并执行脚本，继续解析。
- **优化建议**：将<script>标签放在</body>前，减少阻塞。

4. 从编译原理的角度学习基本语法

- **词法分析**：将代码分解为词法单元（tokens）。
- **语法分析**：构建语法树。
- **语义分析**：理解代码含义，执行指令。

5. 理解JavaScript的编译过程

- **步骤**：
 1. 词法分析和语法分析生成语法树。
 2. 解释执行或即时编译（JIT）优化性能。

6. JavaScript引擎

- **功能**：读取、解析、执行JavaScript代码。
- **现代引擎**：采用JIT编译提升执行速度，如V8（Chrome）、SpiderMonkey（Firefox）。

7. JavaScript试验环境

- **浏览器控制台**：开发者工具中的控制台，用于运行和测试JavaScript代码。

```
console.log('Hello from Console!');
```

8. JS主要内容

- **基本语法**：变量声明、函数定义。
- **标准库**：内置对象和方法。
- **语句和表达式**：控制流、运算符。
- **变量**：var、let、const。
- **条件语句**：if、else、switch。

- **循环语句**: `for`、`while`、`do...while`。
- **数据类型**: 数值、字符串、布尔值、`undefined`、`null`、对象。
- **运算符**: 算术、比较、逻辑、位运算。

9. 变量

- **定义**:

```
var name = 'Alice';  
let age = 25;  
const PI = 3.14159;
```

- **特性**:
 - **动态类型**: 变量类型可随时改变。
 - **作用域**: `var`为函数作用域, `let`和`const`为块级作用域。

10. 条件语句

- **if**:

```
if (score > 90) {  
  console.log('优秀');  
}
```

- **if...else**:

```
if (age >= 18) {  
  console.log('成年人');  
} else {  
  console.log('未成年人');  
}
```

- **switch**:

```
switch(day) {  
  case 'Monday':  
    console.log('星期一');  
    break;
```

```
case 'Tuesday':  
  console.log('星期二');  
  break;  
default:  
  console.log('其他日');  
}
```

11. 循环语句

- **while:**

```
let i = 0;  
while (i < 5) {  
  console.log(i);  
  i++;  
}
```

- **for:**

```
for (let i = 0; i < 5; i++) {  
  console.log(i);  
}
```

- **do...while:**

```
let j = 0;  
do {  
  console.log(j);  
  j++;  
} while (j < 5);
```

- **break 和 continue:**

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) break;  
  if (i % 2 === 0) continue;  
  console.log(i);  
}
```

12. 数据类型简介

- 数值：

```
let num = 100;
```

- 字符串：

```
let str = "Hello";
```

- 布尔值：

```
let isActive = true;
```

- **undefined**：

```
let x;  
console.log(x); // undefined
```

- **null**：

```
let y = null;
```

- 对象：

```
let person = { name: 'Bob', age: 30 };
```

13. **typeof**运算符

- 用法：

```
typeof 123; // "number"  
typeof 'abc'; // "string"
```

```
typeof true; // "boolean"
typeof undefined; // "undefined"
typeof null; // "object"
typeof {}; // "object"
typeof function() {}; // "function"
```

14. null和undefined

- **null**: 表示“无值”，常用于变量初始化。

```
let data = null;
```

- **undefined**: 表示“缺少值”，变量未赋值时自动为**undefined**。

```
let value;
console.log(value); // undefined
```

15. 布尔值

- 值: **true** 和 **false**。
- 应用:

```
let isLoggedIn = true;
if (isLoggedIn) {
  console.log('欢迎用户');
}
```

第十一讲

1. 数值

- 单一数值类型: 基于IEEE 754双精度64位。

```
let num = 42;  
let float = 3.14;
```

- **数值表示法：**

```
let hex = 0xFF; // 255  
let binary = 0b1010; // 10  
let octal = 0o744; // 484  
let scientific = 1.23e4; // 12300
```

- **特殊数值：**

```
let posInf = +Infinity;  
let negInf = -Infinity;  
let notANumber = NaN;
```

- **Number对象：**

```
console.log(Number.MAX_VALUE); // 最大值  
console.log(Number.MIN_VALUE); // 最小值
```

- **安全整数检查：**

```
Number.isSafeInteger(9007199254740991); // true  
Number.isSafeInteger(9007199254740992); // false  
Number.isSafeInteger(100); // true  
Number.MAX_SAFE_INTEGER; // 9007199254740991
```

- **浮点数精度：**

```
console.log(0.1 + 0.2 === 0.3); // false
```

- **数值转换：**

```
let num = parseInt('42'); // 42  
let float = parseFloat('3.14'); // 3.14
```

2. 字符串

- 定义与访问:

```
let greeting = "Hello";  
console.log(greeting[0]); // 'H'
```

- 字符串方法:

```
let text = "  Hello World  ";  
console.log(text.trim()); // "Hello World"  
console.log(text.toUpperCase()); // "  HELLO WORLD  "  
console.log(text.indexOf('World')); // 8  
console.log(text.slice(2, 7)); // "Hello"
```

3. 对象

- 定义与属性:

```
let car = {  
  brand: 'Toyota',  
  model: 'Corolla',  
  start: function() {  
    console.log('Car started');  
  }  
};
```

- 属性访问与赋值:

```
console.log(car.brand); // 'Toyota'  
car.model = 'Camry';
```

- 删除属性:

```
delete car.start;
```

- 属性遍历:

```
for (let key in car) {  
  console.log(key, car[key]);  
}
```

4. 函数

- 声明与调用：

```
function greet(name) {  
  return `Hello, ${name}!`;  
}  
console.log(greet('Alice')); // 'Hello, Alice!'
```

- 函数提升：

```
console.log(add(2, 3)); // 5  
function add(a, b) { return a + b; }
```

- 函数属性：

```
function multiply(a, b) { return a * b; }  
console.log(multiply.name); // 'multiply'  
console.log(multiply.length); // 2
```

- **toString**方法：

```
console.log(multiply.toString());  
// "function multiply(a, b) { return a * b; }"
```

5. 作用域

- 函数作用域：

```
let globalVar = 'I am global';
```



```
function testScope() {  
  let localVar = 'I am local';  
  console.log(globalVar); // 'I am global'  
  console.log(localVar); // 'I am local'  
}  
  
testScope();  
console.log(localVar); // ReferenceError
```

- 变量提升:

```
function hoistExample() {  
  console.log(x); // undefined  
  var x = 10;  
}  
hoistExample();
```

第十二讲

1. 函数名的提升

- 函数声明提升:

```
sayHello(); // 'Hello!'  
  
function sayHello() {  
  console.log('Hello!');  
}
```

- 函数表达式不提升:

```
greet(); // TypeError: greet is not a function  
  
var greet = function() {  
  console.log('Hi!');  
};
```

2. 函数的name属性

- 示例:

```
function originalFunction() {}  
let assignedFunction = originalFunction;  
console.log(originalFunction.name); // 'originalFunction'  
console.log(assignedFunction.name); // 'originalFunction'  
  
let anonFunction = function() {};  
console.log(anonFunction.name); // 'anonFunction'
```

3. 函数的length属性

- 示例:

```
function example(a, b, c) {}  
console.log(example.length); // 3  
  
function partial(a, b) {}  
console.log(partial.length); // 2
```

4. 函数的toString方法

- 示例:

```
function add(a, b) {  
  return a + b;  
}  
console.log(add.toString());  
/*  
function add(a, b) {  
  return a + b;  
}  
*/
```

5. 函数作用域

- 示例:

```
let globalVar = 'global';

function outerFunction() {
  let outerVar = 'outer';

  function innerFunction() {
    let innerVar = 'inner';
    console.log(globalVar); // 'global'
    console.log(outerVar); // 'outer'
    console.log(innerVar); // 'inner'
  }

  innerFunction();
  console.log(innerVar); // ReferenceError
}

outerFunction();
```

6. 函数参数

- 示例：

```
function greet(name, greeting) {
  console.log(`${greeting}, ${name}!`);
}

greet('Alice'); // 'undefined, Alice!'
console.log(greet.length); // 2
```

7. 参数的两种传递方式

- 原始类型（传值）：

```
function changeValue(x) {
  x = 10;
}

let a = 5;
changeValue(a);
console.log(a); // 5
```

- 复合类型（传址）：

```
function changeObject(obj) {  
  obj.name = 'Bob';  
}  
let person = { name: 'Alice' };  
changeObject(person);  
console.log(person.name); // 'Bob'
```

8. arguments对象

- 示例:

```
function sum() {  
  let total = 0;  
  for (let i = 0; i < arguments.length; i++) {  
    total += arguments[i];  
  }  
  return total;  
}  
console.log(sum(1, 2, 3, 4)); // 10
```

9. 数组的定义和本质

- 定义:

```
let fruits = ['苹果', '香蕉', '橘子'];
```

- 本质: 数组是特殊的对象, 索引为键名, 值为数组元素。

10. 数组的length属性

- 示例:

```
let arr = [1, 2, 3];  
console.log(arr.length); // 3  
arr.length = 5;  
console.log(arr); // [1, 2, 3, <2 empty items>]
```

11. 数组的遍历

- 推荐方法：

- **for**循环：

```
for (let i = 0; i < arr.length; i++) {  
  console.log(arr[i]);  
}
```

- **forEach**方法：

```
arr.forEach(function(item) {  
  console.log(item);  
});
```

12. 数组的空位

- 示例：

```
let arr = [1, , 3];  
console.log(arr.length); // 3  
console.log(arr[1]); // undefined
```

13. 运算符

- 算术运算符：+, -, *, /, %, ++, --
- 比较运算符：==, ===, !=, !==, >, <, >=, <=
- 布尔运算符：&&, ||, !
- 位运算符：&, |, ^, ~, <<, >>, >>>
- 其他运算符：?: (三元运算符)、typeof、instanceof

14. 数据类型的转换

- 自动转换：

```
console.log('5' + 1); // '51'
console.log('5' - 1); // 4
```

- 显式转换：

```
let num = Number('42'); // 42
let str = String(100); // '100'
let bool = Boolean(0); // false
```

15. 标准内置对象

- 常用对象：Object, Array, String, Number, Boolean, Function, Date, RegExp, Math
- 示例：

```
let now = new Date();
let regex = /abc/;
let arr = [1, 2, 3];
```

第十三讲

1. URI编解码函数

- 编码和解码：

```
let uri = 'https://www.example.com/?search=JavaScript & CSS';
let encodedURI = encodeURIComponent(uri);
let decodedURI = decodeURIComponent(encodedURI);

let component = 'JavaScript & CSS';
let encodedComponent = encodeURIComponent(component);
let decodedComponent = decodeURIComponent(encodedComponent);

console.log(encodedURI);
// "https://www.example.com/?search=JavaScript%20&%20CSS"
console.log(encodedComponent);
// "JavaScript%20%26%20CSS"
```

2. 字符串对象String

- 方法示例:

```
let str = "Hello, World!";
console.log(str.match(/World/)); // ["World"]
console.log(str.search(/World/)); // 7
console.log(str.replace("World", "JavaScript")); // "Hello, JavaScript!"
console.log(str.charAt(0)); // 'H'
console.log(str.concat(" How are you?")); // "Hello, World! How are you?"
console.log(str.slice(7, 12)); // "World"
console.log(str.indexOf('o')); // 4
console.log(str.lastIndexOf('o')); // 8
console.log(str.trim()); // "Hello, World!"
console.log(str.toLowerCase()); // "hello, world!"
console.log(str.toUpperCase()); // "HELLO, WORLD!"
console.log(str.split(', ')); // ["Hello", "World!"]
console.log('apple'.localeCompare('banana')); // -1
```

3. 日期对象Date

- 创建和方法:

```
let now = new Date();
console.log(now); // 当前日期和时间
let timestamp = Date.now();
console.log(timestamp); // 当前时间的毫秒数
let parsedDate = Date.parse('2024-12-19');
console.log(parsedDate); // 毫秒数
let utcDate = Date.UTC(2024, 11, 19);
console.log(utcDate); // UTC时间的毫秒数
// 获取和设置日期
console.log(now.getFullYear()); // 2024
now.setFullYear(2025);
console.log(now.getFullYear()); // 2025
```

4. 数组对象Array

- 方法示例:

```
let arr = [1, 2, 3];
console.log(Array.isArray(arr)); // true
arr.push(4);
```

```
console.log(arr); // [1, 2, 3, 4]
let last = arr.pop();
console.log(last); // 4
console.log(arr); // [1, 2, 3]
arr.shift();
console.log(arr); // [2, 3]
arr.unshift(1);
console.log(arr); // [1, 2, 3]
let joined = arr.join('-');
console.log(joined); // "1-2-3"
let newArr = arr.concat([4, 5]);
console.log(newArr); // [1, 2, 3, 4, 5]
arr.reverse();
console.log(arr); // [3, 2, 1]
let sliced = arr.slice(1, 3);
console.log(sliced); // [2, 1]
arr.splice(1, 1, 4, 5);
console.log(arr); // [3, 4, 5, 1]
arr.sort();
console.log(arr); // [1, 3, 4, 5]
let mapped = arr.map(x => x * 2);
console.log(mapped); // [2, 6, 8, 10]
arr.forEach(x => console.log(x)); // 1, 3, 4, 5
let filtered = arr.filter(x => x > 3);
console.log(filtered); // [4, 5]
let some = arr.some(x => x > 4);
console.log(some); // true
let every = arr.every(x => x > 0);
console.log(every); // true
let reduced = arr.reduce((acc, val) => acc + val, 0);
console.log(reduced); // 13
```

5. Math对象

- 常用方法：

```
console.log(Math.PI); // 3.141592653589793
console.log(Math.random()); // 0到1之间的随机数
console.log(Math.floor(4.7)); // 4
console.log(Math.ceil(4.2)); // 5
console.log(Math.round(4.5)); // 5
console.log(Math.max(1, 3, 2)); // 3
console.log(Math.min(1, 3, 2)); // 1
console.log(Math.pow(2, 3)); // 8
console.log(Math.sqrt(16)); // 4
console.log(Math.sin(Math.PI / 2)); // 1
```

6. JavaScript与DOM

- **定义：**DOM是文档对象模型，表示网页的结构，允许通过JavaScript动态操作。
- **DOM操作示例：**

```
// 访问元素
let header = document.getElementById('header');
// 修改内容
header.textContent = '新的标题';
// 创建元素
let newDiv = document.createElement('div');
newDiv.textContent = '新内容';
// 插入元素
document.body.appendChild(newDiv);
// 删除元素
let oldElement = document.getElementById('old');
oldElement.remove();
```

第十四讲

DOM节点操作

- **创建节点：**

```
let div = document.createElement('div');
let text = document.createTextNode('Hello DOM');
div.appendChild(text);
```

- **插入节点：**

```
document.body.appendChild(div);
```

- **删除节点：**

```
document.body.removeChild(div);
```

- **复制和替换节点：**

```
let clone = div.cloneNode(true);
document.body.replaceChild(clone, div);
```

- 节点关系检查:

```
console.log(document.body.contains(clone)); // true
```

DOM节点接口和对象

- Node接口:

- 属性: nodeName,.nodeType, parentNode, childNodes。
- 方法: appendChild(), removeChild(), replaceChild(), cloneNode()。

- NodeList和HTMLCollection:

```
let elements = document.querySelectorAll('.item'); // NodeList
let liveElements = document.getElementsByTagName('div'); // HTMLCollection
```

- ParentNode和ChildNode接口:

- ParentNode: append(), prepend(), before(), after()。
- ChildNode: remove(), replaceWith()。

- Document节点对象:

- 属性: title, URL, body, head。
- 方法: getElementById(), querySelector(), createElement()。

- Element对象:

- 属性: innerHTML, style, classList。
- 方法: getAttribute(), setAttribute(), classList.add()。

- Text和DocumentFragment节点:

- Text节点: 包含文本内容。
- DocumentFragment: 用于批量操作, 提高性能。

事件

- **事件定义：** 用户操作（点击、输入）或浏览器行为（加载完成）触发的动作。
- **事件模型：** 基于事件监听，响应事件。
- **事件操作：**

- **添加监听：**

```
element.addEventListener('click', function(event) {  
    console.log('元素被点击');  
});
```

- **移除监听：**

```
function handler(event) {  
    console.log('事件处理');  
}  
element.addEventListener('click', handler);  
element.removeEventListener('click', handler);
```

- **派发事件：**

```
let event = new Event('build');  
element.dispatchEvent(event);
```

- **事件传播：**
 - **捕获阶段：** 事件从根节点向目标节点传播。
 - **目标阶段：** 事件到达目标节点。
 - **冒泡阶段：** 事件从目标节点向根节点传播。

- **事件代理：**

```
document.getElementById('parent').addEventListener('click', function(event) {  
    if (event.target && event.target.matches('button.class')) {  
        console.log('按钮被点击');  
    }  
});
```

- 事件停止传播：

```
element.addEventListener('click', function(event) {  
  event.stopPropagation();  
});
```

事件类型

- 鼠标事件：

- 常用：click, dblclick, mousedown, mouseup, mousemove.
- 示例：

```
element.addEventListener('click', () => {  
  console.log('单击');  
});
```

- 键盘事件：

- 常用：keydown, keypress, keyup.
- 示例：

```
document.addEventListener('keydown', (event) => {  
  console.log(`按键 ${event.key} 被按下`);  
});
```

- 表单事件：

- 常用：input, select, change, invalid, reset, submit.
- 示例：

```
document.querySelector('form').addEventListener('submit', (event) => {  
  event.preventDefault();  
  console.log('表单提交');  
});
```